



UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA · ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS

Patrones de Diseño, principios SOLID y frameworks

Análisis y Diseño de Sistemas 2 · Sección B — Semana 2

Abdiel Fernando José Otzoy Oztín

Fecha: Jueves 23 de julio de 2026 Hora: 17:20

Agenda de hoy

01

Lectura de la Práctica 1

Lineamientos, alcance y entrega de la primera práctica.

02

Patrones de diseño

Qué son, por qué importan y cómo se clasifican.

03

Principios SOLID

Cinco reglas para escribir clases mantenibles y extensibles.

04

Frameworks y librerías

La diferencia clave: quién controla el flujo del programa.

05

Foro de la semana

Discusión guiada: aplicar lo visto a un caso real.

06

Dudas y comentarios

Espacio abierto de preguntas.

Competencia que desarrollaremos

“ Diseñar y aplicar patrones de diseño de software utilizando principios de diseño y catálogos de estilos, garantizando la calidad estructural y de comportamiento del sistema.

¿QUÉ SIGNIFICA ESTO EN LA PRÁCTICA?



Reconocer problemas recurrentes

Identificar cuándo un problema de diseño ya fue resuelto y nombrar el patrón adecuado.



Justificar decisiones de diseño

Usar los principios SOLID como argumento técnico, no como gusto personal.



Evaluar y refactorizar código

Detectar violaciones a los principios y proponer una solución concreta y explicable.

SECCIÓN 01

Lectura de la Práctica 1

01

Antes de entrar al tema, revisaremos juntos el enunciado de la primera práctica: qué se pide, cómo se entrega y cómo se evalúa.

Objetivo y alcance de la práctica

Requisitos funcionales y restricciones

Criterios de evaluación

Fecha y forma de entrega

SECCIÓN 02

Patrones de Diseño

02

Soluciones comprobadas y documentadas para problemas que se repiten en el diseño de software orientado a objetos.

Qué es un patrón y qué es un patrón de diseño

Por qué vale la pena usarlos

Cómo se documenta un patrón (formato estándar)

El catálogo GoF: creacionales, estructurales y de comportamiento

¿Qué es un patrón?

Un **patrón** es un modelo, regla o estructura **repetitiva** que se observa en objetos, eventos o datos: una regularidad predecible que podemos identificar y reutilizar para comprender o resolver situaciones parecidas.

Los patrones no son exclusivos del software: los usamos todos los días sin darnos cuenta.



Patrón de costura

Una plantilla de papel que guía cómo cortar la tela: la misma plantilla produce muchas prendas iguales.



Plano arquitectónico

La misma distribución de una casa modelo se repite en cientos de viviendas con pequeños ajustes.



Receta de cocina

Pasos probados que cualquiera puede seguir y adaptar (más sal, menos picante) sin inventar el platillo de nuevo.



Idea clave: en software, un patrón captura una solución que ya funcionó, para no tener que reinventarla en cada proyecto.

¿Qué es un patrón de diseño?

“ Un patrón de diseño es una descripción de objetos y clases que se comunican y están personalizados para resolver un problema general de diseño en un contexto específico.

Design Patterns: Elements of Reusable Object-Oriented Software — Gamma, Helm, Johnson, Vlissides (“*Gang of Four*”, *GoF*) · 1994

“ Un patrón de diseño es una solución comprobada a un problema común en un contexto específico. No es código, sino una guía para resolver un problema de diseño que se repite en el desarrollo de software.

Head First Design Patterns — Freeman, Robson, Bates, Sierra · 2004

Lo que ambas definiciones coinciden: un patrón no es una función ni una librería lista para copiar. Es una **plantilla de solución reutilizable** que cada equipo adapta a su propio contexto y lenguaje.

¿Por qué vale la pena usarlos?



Reutilización

Resuelven problemas comunes con soluciones ya probadas y eficientes.



Lenguaje común

Decir “aquí va un Observer” comunica una arquitectura completa en dos palabras.



Mantenibilidad

El código queda más modular, legible y fácil de modificar.



Escalabilidad

Facilitan agregar funcionalidades nuevas sin romper el sistema.



Diseño limpio

Promueven la separación de responsabilidades y un bajo acoplamiento.



Conocimiento colectivo

Destilan décadas de experiencia de la comunidad de desarrollo.

En resumen: diseñar con patrones es aprender de la experiencia acumulada en lugar de empezar desde cero.

¿Cómo se documenta un patrón?

Los catálogos (GoF, refactoring.guru, bibliografía del curso) describen cada patrón con una **ficha estándar**. Conocerla les permite leer cualquier patrón nuevo por su cuenta.

01 Nombre

Identifica al patrón en una o dos palabras; se vuelve parte del vocabulario del equipo.

02 Propósito / Intención

Qué problema resuelve y en una frase: qué hace el patrón.

03 Problema y contexto

Cuándo aplicarlo: las condiciones que deben darse para que tenga sentido.

04 Solución

Las clases y objetos que participan, sus responsabilidades y cómo colaboran.

05 Participantes

El rol de cada clase dentro de la estructura del patrón.

06 Consecuencias

Ventajas y desventajas: qué se gana y qué se sacrifica al aplicarlo.

07 Ejemplo de código

Una implementación de referencia en un lenguaje concreto.

08 Usos conocidos

Sistemas reales o frameworks donde ya aparece aplicado.

El catálogo GoF: tres familias

El catálogo original (GoF, 1994) reúne **23 patrones**, agrupados según el tipo de problema que resuelven.

Creacionales

¿Cómo se crean los objetos?

Abstraen el proceso de instanciación para que el sistema no dependa de clases concretas.

Singleton
Factory Method
Abstract Factory
Builder
Prototype

Estructurales

¿Cómo se componen las clases?

Definen cómo armar estructuras flexibles a partir de clases y objetos existentes.

Adapter
Bridge
Composite
Decorator
Facade
Proxy

De comportamiento

¿Cómo colaboran los objetos?

Distribuyen responsabilidades y definen cómo se comunican los objetos entre sí.

Observer
Strategy
Command
Iterator
State
Template Method

En este curso trabajaremos varios de ellos; hoy basta con ubicar cada familia y un ejemplo de cada una.

Antes de los patrones: principios de diseño

¿QUÉ SON?

Reglas y buenas prácticas que guían el diseño orientado a objetos para construir software **modular, mantenible, reutilizable y fácil de entender**. Son la base sobre la que se apoyan los patrones de diseño.

¿POR QUÉ IMPORTAN?

- 1 Organizan el código de forma coherente y flexible.
- 2 Reducen la complejidad del software a medida que crece.
- 3 Facilitan la colaboración en equipos grandes.

LOS QUE USAREMOS EN EL CURSO

Encapsular lo que varía

Aislar las partes que cambian de las que permanecen estables.

Programar contra interfaces

Depender de abstracciones, no de implementaciones concretas.

Favorecer composición sobre herencia

Armar comportamiento combinando objetos, no heredando.

Alta cohesión y bajo acoplamiento

Cada pieza hace una cosa y depende de pocas más.

SOLID

Cinco principios que veremos a detalle en la siguiente sección.

SECCIÓN 03

Principios SOLID

03

Cinco reglas de diseño orientado a objetos, recopiladas por Robert C. Martin, que hacen el código más comprensible, flexible y mantenible.

S — Responsabilidad única (Single Responsibility)

O — Abierto / cerrado (Open-Closed)

L — Sustitución de Liskov (Liskov Substitution)

I — Segregación de interfaces (Interface Segregation)

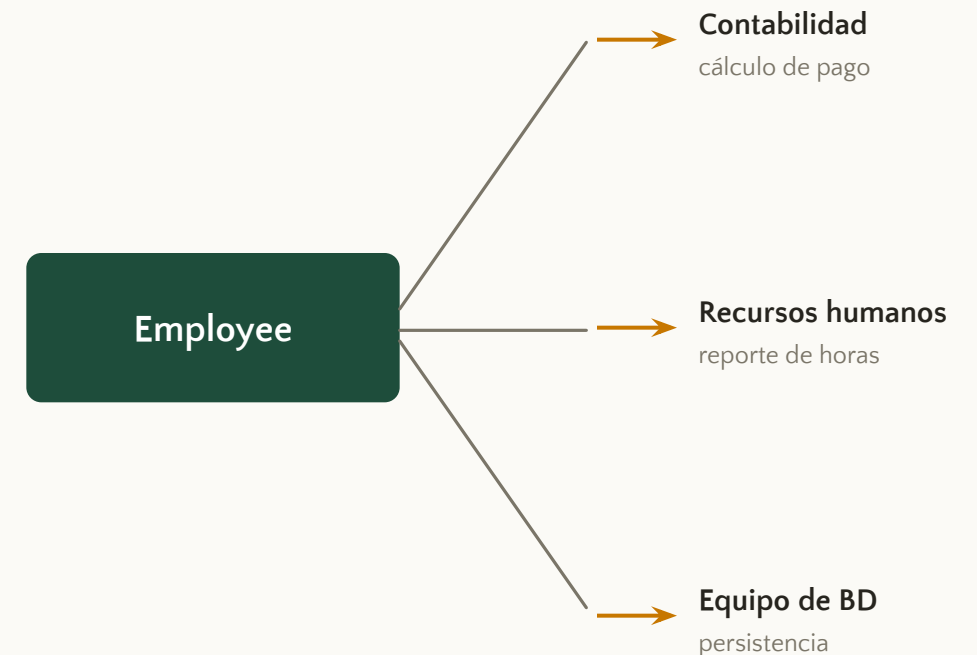
D — Inversión de dependencias (Dependency Inversion)

SRP · Una clase, una sola razón para cambiar

El **Principio de Responsabilidad Única** establece que una clase debe tener **una sola razón para cambiar**: encargarse de una única responsabilidad dentro del sistema.

Una “razón para cambiar” suele ser **un actor del negocio** que puede pedir modificaciones: contabilidad, recursos humanos, el equipo de base de datos...

Analogía: un empleado que atiende la caja, cocina y limpia el local tiene tres jefes distintos; cualquiera de ellos puede cambiarle las tareas y afectar las demás. Con un solo puesto bien definido, los cambios son locales y predecibles.



Tres actores distintos piden cambios a la misma clase: tres razones para cambiar. Se viola SRP.

SRP en riesgo: una clase que lo hace todo

¿QUÉ ESTÁ MAL AQUÍ?

La clase **Employee** acumula **tres responsabilidades** que responden a tres actores distintos.

Si Contabilidad cambia la fórmula del pago, hay que modificar la misma clase que usan RRHH y el equipo de BD: un error al editar puede romper funciones que no tenían nada que ver.

Además, los cambios de tres equipos se cruzan en un solo archivo: conflictos de versión y pruebas que repetir cada vez.

✗ VIOLA EL PRINCIPIO

```
class Employee {  
  
    // 1) Cálculo de salario → le interesa a  
    Contabilidad  
    double calculatePay() { ... }  
  
    // 2) Reporte de horas → le interesa a RRHH  
    String reportHours() { ... }  
  
    // 3) Persistencia → le interesa al equipo de BD  
    void save() { ... }  
}
```

Señal de alerta: si al describir una clase usas “y” varias veces (“calcula el salario y reporta horas y se guarda sola”), probablemente tiene más de una razón para cambiar.

SRP aplicado: una responsabilidad por clase

Separamos cada responsabilidad en su propia clase: **PayCalculator**, **HoursReporter** y **EmployeeRepository**.

Ahora, si cambia la fórmula del salario, solo se toca **PayCalculator**; los reportes y la persistencia quedan intactos.

La clase **Employee** se reduce a lo que realmente es: los datos del empleado.

Beneficio inmediato: cada equipo prueba y despliega su parte sin pisar el trabajo de los demás.

✓ SOLUCIÓN CORRECTA

```
// Cada clase responde a un solo actor
class PayCalculator {
    double calculatePay(Employee e) { ... }
}

class HoursReporter {
    String reportHours(Employee e) { ... }
}

class EmployeeRepository {
    void save(Employee e) { ... }
}
```

Regla práctica: nombra la responsabilidad de cada clase en una frase. Si necesitas más de una frase, la clase está pidiendo a gritos dividirse.

OCP · Abierto para extensión, cerrado para modificación

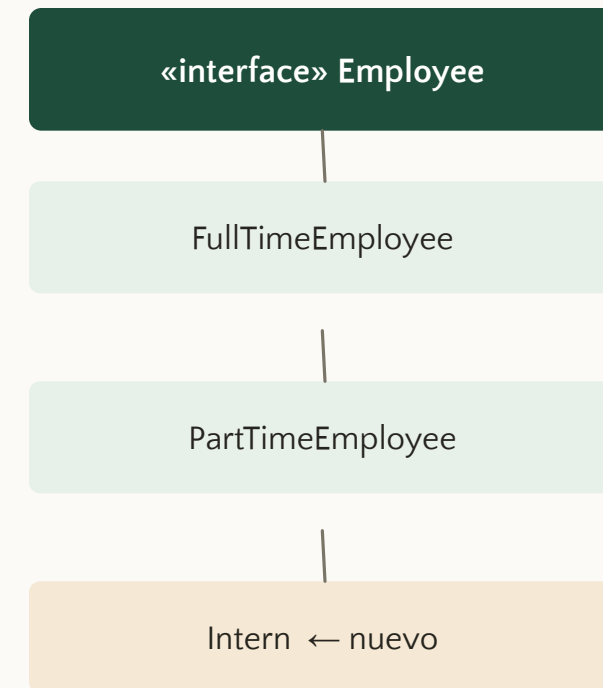
El **Principio Abierto/Cerrado** establece que las entidades de software (clases, módulos, funciones) deben estar **abiertas para extensión**, pero **cerradas para modificación**.

En otras palabras: para agregar un comportamiento nuevo debería bastar con **escribir código nuevo**, sin editar el código que ya funciona y ya fue probado.

¿CÓMO SE LOGRA?

Definiendo **abstracciones estables** (interfaces o clases abstractas) y permitiendo que los comportamientos nuevos lleguen como **implementaciones nuevas**: herencia y polimorfismo al servicio de la extensión.

Cada cambio que obliga a editar una clase ya estable es una señal de que falta una abstracción.



El comportamiento nuevo (**Intern**) llega como una clase nueva: el código existente no se toca.

OCP en riesgo: extender editando lo que ya funciona

¿QUÉ ESTÁ MAL AQUÍ?

La cadena de **if/else** concentra todos los casos en un solo método.

Cada vez que aparezca un tipo nuevo de empleado (contratista, becario, por comisión...) hay que **abrir y modificar** la misma clase — que ya estaba probada y en producción.

Editar lo que ya funciona implica volver a probar todos los casos anteriores y arriesgar errores en tipos que nadie quería tocar.

✗ VIOLA EL PRINCIPIO

```
class SalaryCalculator {  
    double calculate(String type, int hours, double rate) {  
        if (type.equals("FULL_TIME")) return hours * rate;  
        if (type.equals("PART_TIME")) return hours * rate *  
0.8;  
  
        if (type.equals("INTERN")) return hours * rate * 0.5;  
        // ¿Un tipo nuevo? → toca MODIFICAR esta clase  
        throw new IllegalArgumentException("Tipo  
desconocido");  
    }  
}
```

Señal de alerta: cadenas de **if/else** o **switch** sobre un “tipo” que crecen con cada requisito nuevo casi siempre delatan una violación de OCP.

OCP aplicado: extender sin modificar

Extraemos la operación variable a una **interfaz Employee** con un único contrato: **calculatePay()**.

Cada tipo de empleado es una clase que implementa su propia fórmula. El código que los usa solo conoce la interfaz:

for (Employee e : employees) → e.calculatePay()

Cuando llega un tipo nuevo, se **agrega una clase nueva**: las existentes no se modifican ni se vuelven a probar.

✓ SOLUCIÓN CORRECTA

```
interface Employee {  
    double calculatePay();  
}  
  
class FullTimeEmployee implements Employee {  
    public double calculatePay() { return hours * rate; }  
}  
  
// ¿Tipo nuevo? Se AGREGA una clase, no se edita nada:  
class Intern implements Employee {  
    public double calculatePay() { return hours * rate * 0.5;  
}  
}
```

Regla práctica: donde el requisito cambie seguido (“pueden aparecer más tipos”), pon una abstracción; así el cambio llega como código nuevo, no como cirugía al código viejo.

LSP · Las subclases deben sustituir a su clase base

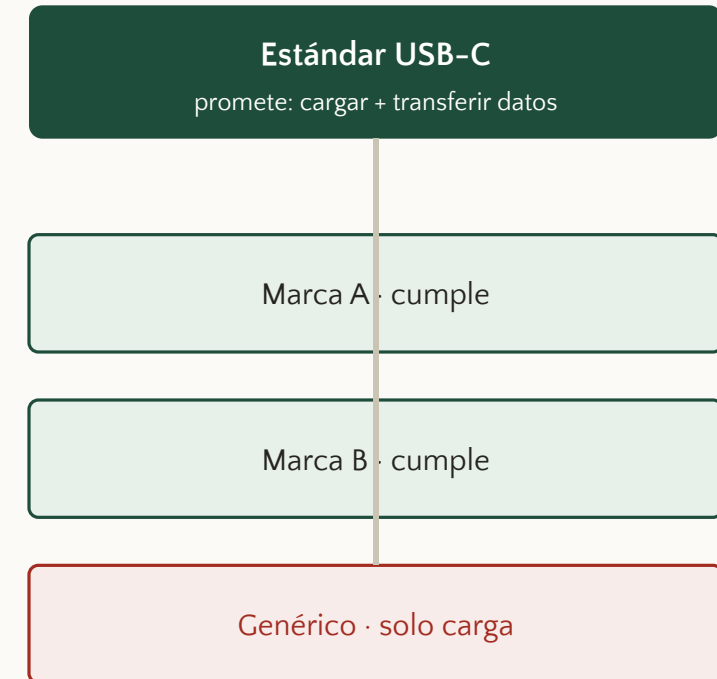
El **Principio de Sustitución de Liskov** establece que los objetos de una clase derivada deben poder **sustituir a los de su clase base sin alterar el comportamiento correcto del programa**.

ANALOGÍA: EL CABLE USB-C

El **USB-C es un estándar**: define lo que cualquier cable debe cumplir (cargar, transferir datos, encajar en el puerto). Cada marca lo **implementa** a su manera —Anker, Samsung, el genérico— pero tú deberías poder tomar **cualquier cable** y que funcione como esperas.

Si un cable “USB-C” cargara pero **no transfiriera datos**, rompería lo que el estándar promete: dejarías de confiar en cualquier cable.

Igual en el código: la clase base es el estándar y cada subclase es una marca. Una subclase que no se comporta como lo esperado **rompe este principio**.



Todas son “USB-C”, pero la tercera no cumple el estándar → no puede sustituirlo sin romper tu confianza.

LSP con pingüinos: no todo pájaro vuela

❌ VIOLA EL PRINCIPIO

```
class Bird {
    void fly() { /* volar */ }
    void eat() { ... }
}

class Penguin extends Bird {
    // Un pingüino NO vuela:
    // hereda una promesa que no puede cumplir
    @Override void fly() {
        throw new UnsupportedOperationException();
    }
}
```

Bird promete que todo pájaro vuela. **Penguin** hereda esa promesa, pero no puede cumplirla: cualquier código que haga **bird.fly()** explotará si recibe un pingüino. La subclase ya no sustituye a la base → se viola LSP.

✅ SOLUCIÓN CORRECTA

```
// Bird solo promete lo que TODOS cumplen
class Bird {
    void eat() { ... }
}

// Volar es una capacidad aparte:
interface Flyable { void fly(); }

class Sparrow extends Bird
    implements Flyable { void fly() { ... } }

class Penguin extends Bird {
    // nada que fingir: no promete volar
}
```

Volar sale de Bird y pasa a la interfaz **Flyable**: solo los pájaros que realmente vuelan la implementan. Nadie hereda promesas que no puede cumplir.



Regla práctica: la herencia no es “es un” en la vida real, sino “se comporta como” en el código. Si la subclase anula métodos lanzando excepciones, la jerarquía está mal planteada.

ISP · Interfaces pequeñas y específicas

El **Principio de Segregación de Interfaces** establece que una clase **no debe verse obligada a depender ni a implementar métodos que no utiliza**.

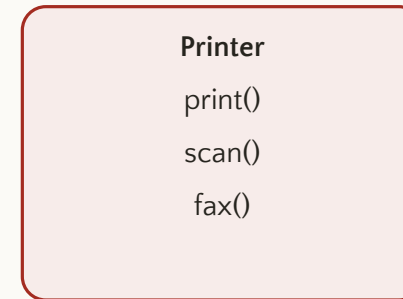
Es preferible diseñar **interfaces pequeñas y específicas** en lugar de interfaces grandes y generales, de modo que cada cliente dependa solo de lo que realmente necesita.

SEÑALES DE UNA INTERFAZ “GORDA”

- ⚠ Métodos implementados lanzando `UnsupportedOperationException`.
- ⚠ Implementaciones vacías o con comentarios “aquí no aplica”.
- ⚠ Cambiar la interfaz obliga a recompilar clases que no la usan completa.

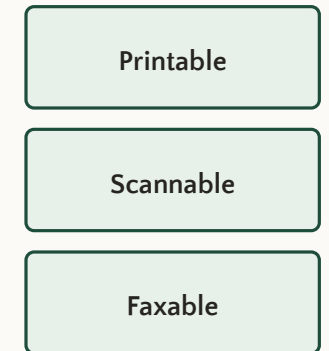
Idea clave: segregar no es multiplicar interfaces por gusto; es que cada cliente reciba exactamente el contrato que necesita — ni más, ni menos.

UNA INTERFAZ PARA TODOS



VS

TRES ESPECÍFICAS



ISP en riesgo: obligar a implementar lo que no se usa

¿QUÉ ESTÁ MAL AQUÍ?

La interfaz **Printer** asume que toda impresora imprime, escanea y envía fax.

Una impresora sencilla se ve **obligada a implementar métodos que no soporta**, y la única salida es lanzar excepciones en tiempo de ejecución.

El contrato miente: quien recibe un **Printer** no puede confiar en que sus métodos funcionen.

✗ VIOLA EL PRINCIPIO

```
interface Printer {  
    void print();  
    void scan();  
    void fax();  
}  
  
class SimplePrinter implements Printer {  
    public void print() { /* ok */ }  
    public void scan() {  
        throw new UnsupportedOperationException("No  
escanea");  
    }  
    public void fax() {  
        throw new UnsupportedOperationException("No envía  
fax");  
    }  
}
```

Señal de alerta: implementaciones que solo lanzan **UnsupportedOperationException** o quedan vacías indican que la interfaz exige más de lo que el cliente necesita.

ISP aplicado: un contrato por capacidad

Dividimos la interfaz “gorda” en tres contratos específicos: **Printable**, **Scannable** y **Faxable**.

La impresora sencilla implementa solo **Printable**; la multifuncional compone las tres interfaces. Nadie implementa métodos que no usa.

Y el código cliente ahora declara exactamente lo que necesita: una función que escanea recibe un **Scannable**, no un Printer completo.

✓ SOLUCIÓN CORRECTA

```
interface Printable { void print(); }
interface Scannable { void scan(); }
interface Faxable { void fax(); }

// Cada equipo implementa SOLO lo que sabe hacer
class SimplePrinter implements Printable { ... }

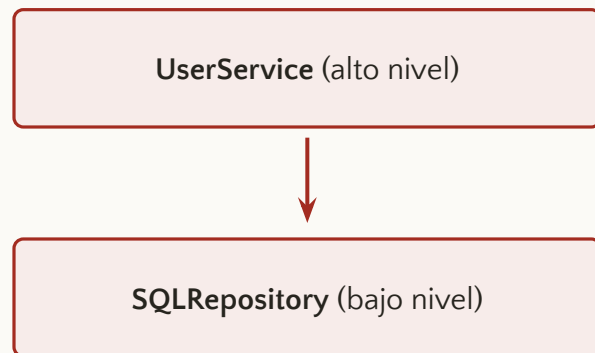
class MultiFunctionPrinter implements
    Printable, Scannable, Faxable { ... }
```

Regla práctica: diseña interfaces desde la mirada del cliente: pregunta “¿qué necesita quien la va a usar?” en lugar de “¿qué podría llegar a hacer alguien algún día?”.

DIP · Dependier de abstracciones, no de implementaciones

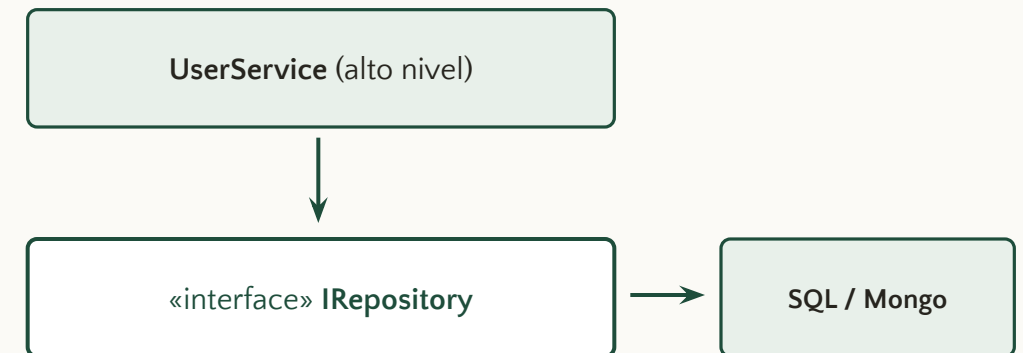
El **Principio de Inversión de Dependencias** establece que los módulos de alto nivel no deben depender de módulos de bajo nivel: **ambos deben depender de abstracciones**. Además, las abstracciones no deben depender de los detalles; los detalles deben depender de las abstracciones.

SIN DIP: DEPENDENCIA DIRECTA



La lógica de negocio queda atada a una base de datos concreta: cambiarla obliga a reescribirla.

CON DIP: AMBOS DEPENDEN DE LA ABSTRACCIÓN



El detalle (SQL, Mongo...) implementa la abstracción; la lógica de negocio ya no conoce la base de datos concreta.

“**Inversión**” porque la dependencia se invierte: ya no es el alto nivel el que conoce al bajo nivel, sino el detalle el que se adapta a la abstracción.

DIP en riesgo: el servicio crea sus propios canales

¿QUÉ ESTÁ MAL AQUÍ?

El **NotificacionService** (lógica de alto nivel) instancia directamente cada canal concreto: **EmailSender**, **SMSSender**...

Cada integración nueva (WhatsApp, Slack, Telegram) obliga a **abrir y modificar la misma clase**: más atributos, más métodos, más riesgo de romper lo que ya funcionaba.

Además, probar el servicio sin enviar correos reales es casi imposible: no hay forma de cambiar el canal desde fuera.

❌ VIOLA EL PRINCIPIO

```
class NotificacionService {  
  
    // El servicio CONSTRUYE cada canal concreto  
    private EmailSender email = new EmailSender();  
    private SMSSender sms = new SMSSender();  
  
    void notificarPorEmail(String msg) { email.send(msg); }  
    void notificarPorSMS(String msg) { sms.send(msg); }  
  
    // ¿WhatsApp, Slack, Telegram...?  
    // Toca MODIFICAR esta clase otra vez  
}
```

Señal de alerta: cuando agregar una integración significa editar la clase de servicio (otro **new**, otro método), el servicio depende de los detalles en lugar de una abstracción.

DIP aplicado: cada integración es una clase nueva

SOLUCIÓN CORRECTA

Extraemos la abstracción **Notificacion** con un único contrato: **enviar(msg)**.

Cada integración es **una clase nueva** que implementa la interfaz:

EmailNotificacion, SMSNotificacion, WhatsAppNotificacion...

El servicio solo conoce la interfaz y recibe el canal por el constructor
(**inyección de dependencias**): agregar Telegram mañana es crear una clase,
sin tocar el servicio.

```
interface Notificacion {  
    void enviar(String msg);  
}  
  
// Cada integración = una clase nueva que la implementa  
class EmailNotificacion implements Notificacion { ... }  
class SMSNotificacion implements Notificacion { ... }  
class WhatsAppNotificacion implements Notificacion { ... }  
  
class NotificacionService {  
    private Notificacion canal;  
  
    // El canal llega desde afuera (inyección)  
    NotificacionService(Notificacion canal) { this.canal = canal; }  
  
    void notificar(String msg) { canal.enviar(msg); }  
}
```

Bonus: este diseño también cumple OCP — el servicio queda cerrado a modificación pero abierto a nuevas integraciones. Los principios SOLID se refuerzan entre sí.

SECCIÓN 04

Frameworks y Librerías

04

Ambos reutilizan código de terceros, pero difieren en algo fundamental: quién tiene el control del flujo del programa.

Qué es una librería

Qué es un framework

La diferencia clave: inversión de control

Cuándo conviene cada uno

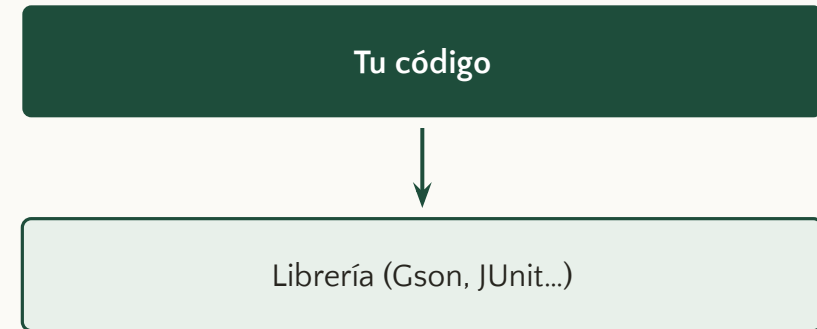
¿Qué es una librería?

Una **librería** es un conjunto de funciones y clases reutilizables que **tu código llama cuando las necesita** para resolver tareas puntuales: convertir JSON, hacer cálculos, formatear fechas...

Tú tienes el control: decides cuándo y cómo usarla.

ANALOGÍA

Una **caja de herramientas**: la abres cuando necesitas un destornillador, lo usas a tu manera y lo guardas. La caja no te dice cómo construir tu mueble.



EJEMPLOS

Gson / Jackson

convertir objetos Java ↔ JSON

Apache Commons

utilidades generales para Java

JUnit

escribir y ejecutar pruebas unitarias

NumPy / Pandas

cálculo numérico y datos en Python

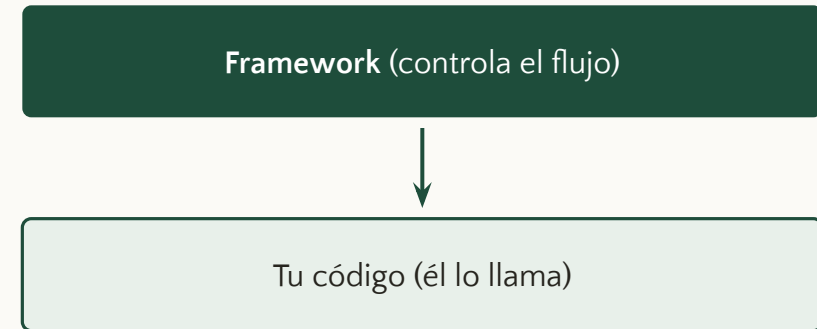
¿Qué es un framework?

Un **framework** es una estructura de trabajo que define el **flujo y el esqueleto de la aplicación**: tú escribes tus clases siguiendo sus reglas y **él las llama cuando corresponde**.

El framework tiene el control del programa; tu código llena los espacios que él deja definidos.

ANALOGÍA

La **planta de un restaurante**: la cocina, los tiempos y el recorrido de los platos ya están definidos; tú solo decides el menú. Si quieres salirte del esquema, cuesta trabajo.



EJEMPLOS

Spring

aplicaciones empresariales en Java

Angular

aplicaciones web en el navegador

Django

aplicaciones web en Python

Flutter

aplicaciones móviles multiplataforma

La diferencia clave: inversión de control

CRITERIO	LIBRERÍA	FRAMEWORK
¿Quién llama a quién?	Tu código llama a la librería	El framework llama a tu código
¿Quién controla el flujo?	Tú, en todo momento	El framework (inversión de control)
¿Qué te da?	Funciones para tareas puntuales	El esqueleto completo de la aplicación
Libertad vs. estructura	Más libertad, más decisiones tuyas	Más estructura y convenciones
Ejemplos	Gson, JUnit, NumPy	Spring, Angular, Django



Principio de Hollywood – “No nos llames; nosotros te llamamos”: esa inversión de control es lo que convierte una colección de librerías en un framework.

Foro de la semana

PREGUNTA PARA DISCUTIR

Piensa en un proyecto (propio, del curso o de tu trabajo) donde exista una clase que “lo hace todo”: ¿qué principio SOLID está violando, qué problemas concretos ha causado y cómo la separarías aplicando lo visto hoy?

CÓMO PARTICIPAR

1

Publica tu análisis

Describe la clase, el principio violado y tu propuesta de separación en el foro del curso.

2

Comenta a un compañero

Responde al menos una publicación: ¿estás de acuerdo con su diagnóstico? ¿Agregarías algo?

3

Trae tus dudas a clase

Los casos más interesantes los retomamos en la próxima sesión.

Dudas y comentarios

¿Qué pregunta se quedó pendiente?

Este es el momento de aclarar lo que no quedó claro antes de la práctica: ninguna duda es menor si te va a ahorrar horas de código mal diseñado.

PARA LLEVARTE HOY

Patrón de diseño

Solución comprobada y documentada a un problema recurrente; no es código, es una guía.

SOLID

Cinco principios para clases con una sola razón de cambio, extensibles sin modificarlas y desacopladas.

Framework ≠ librería

La diferencia está en quién controla el flujo: inversión de control.

¡Gracias por su atención!

Nos vemos la próxima semana para continuar con los patrones del catálogo GoF. No olviden participar en el foro y revisar los lineamientos de la Práctica 1.

Abdiel Fernando José Otzoy Otzín

Universidad de San Carlos de Guatemala · Facultad de Ingeniería · EICS

